Describing them is pretty much a standard process: algorithms are described in terms of the input they take, and the output they provide. Their details can be illustrated with pseudo-code (ignoring implementation details of programming languages), or actual code.

Data structures follows the same conventions, but they also go slightly beyond: we have to describe as well the actions you can perform on a data structure; usually each action is described in term of an algorithm, so with an input, an output, but *in addition* to those, for DSs we also need to describe *side effects*, the changes an action might cause to the data structure itself.

To fully understand what this means, we first need to properly define data structures.

1.2.1 Defining a Data Structure

A Data Structure (DS) is a specific solution of organizing data that provides storage for items, and capabilities[2] for storing and retrieving them.

The simplest example of a DS is an array: for instance, an array of characters provides storage for a finite number of characters, and methods to retrieve each character in the array based on its position. Figure 1.1 shows how `array` = [‘C’, ‘A’, ‘R’] is stored: an array of characters storing the characters `C`, `A`, and `R`, such that f.i. `array[1]` corresponds to the value `‘A’`.